

Task 11: Implement a Data Access Layer using Spring Data JPA. Use JpaRepository and custom query methods to retrieve student records based on conditions such as department or age, and include sorting and pagination

Aim :

To design and implement a Data Access Layer using Spring Data JPA by utilizing JpaRepository to perform CRUD operations and create custom query methods for retrieving student records based on conditions such as department and age, along with implementing sorting and pagination.

Algorithm:

1. Create a Spring Boot project and configure the database.
2. Define a Student entity with id, name, department, and age.
3. Create a repository interface extending JpaRepository.
4. Add custom query methods to fetch students by department and age.
5. Retrieve data from the repository based on given conditions.
6. Apply sorting using Sort (ascending/descending).
7. Apply pagination using PageRequest (pageNumber, pageSize).
8. Extract required records using getContent().
9. Return the processed student data.

Program:

1. Student Entity

```
import jakarta.persistence.*;

@Entity

@Table(name = "students")
public class student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "student_name", nullable = false)
    private String name;

    private int age;

    private String department;
    public student() {}
    public student(String name, int age, String department) {
        this.name = name;
        this.age = age;
        this.department = department; }
}
```

```

    public Long getId() {
        return id;
    }
    public String getName() {
        return name;
    }

    public int getAge() {
        return age;
    }

    public String getDepartment() {
        return department;
    }
    public void setId(Long id) {
        this.id = id;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public void setDepartment(String department) {
        this.department = department;
    }
}

```

2. Repository Interface

```

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.example.demo.student;

@Repository
public interface StudentRepository extends JpaRepository<student, Long> {

    List<student> findByDepartment(String department);

    List<student> findByAgeGreaterThan(int age);

    List<student> findByName(String name);

}

```

3. Service Layer

```
import java.util.List;

import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import com.example.demo.student;
import com.example.demo.repository.StudentRepository;

@Service
public class StudentService {

    private final StudentRepository repository;

    public StudentService(StudentRepository repository) {
        this.repository = repository;
    }

    public student saveStudent(student student) {
        return repository.save(student);
    }

    public List<student> getAllStudents() {
        return repository.findAll();
    }

    public student getStudentById(Long id) {
        return repository.findById(id).orElse(null);
    }

    public student updateStudent(Long id, student student) {
        student existing = repository.findById(id).orElse(null);
        if (existing != null) {
            existing.setName(student.getName());
            existing.setAge(student.getAge());
            existing.setDepartment(student.getDepartment());
        }
    }
}
```

```

        return repository.save(existing);
    }
    return null;
}

public void deleteStudent(Long id) {
    repository.deleteById(id);
}

public List<student> getStudentsByDepartment(String department) {
    return repository.findByDepartment(department); }

public List<student> getStudentsByAgeGreaterThan(int age) {
    return repository.findByAgeGreaterThan(age); }

public List<student> getStudentsSortedByName() {
    return repository.findAll(Sort.by(Sort.Direction.ASC, "name"));
}

public Page<student> getStudentsWithPagination(int page, int size) {
    Pageable pageable = PageRequest.of(page, size);
    return repository.findAll(pageable);
} }

```

4. Controller Layer

```

import java.util.List;
import org.springframework.web.bind.annotation.*;
import com.example.demo.student;
import com.example.demo.service.StudentService;

@RestController
@RequestMapping("/students")
public class StudentController {
    private final StudentService service;

    public StudentController(StudentService service) {
        this.service = service;
    }
    @PostMapping

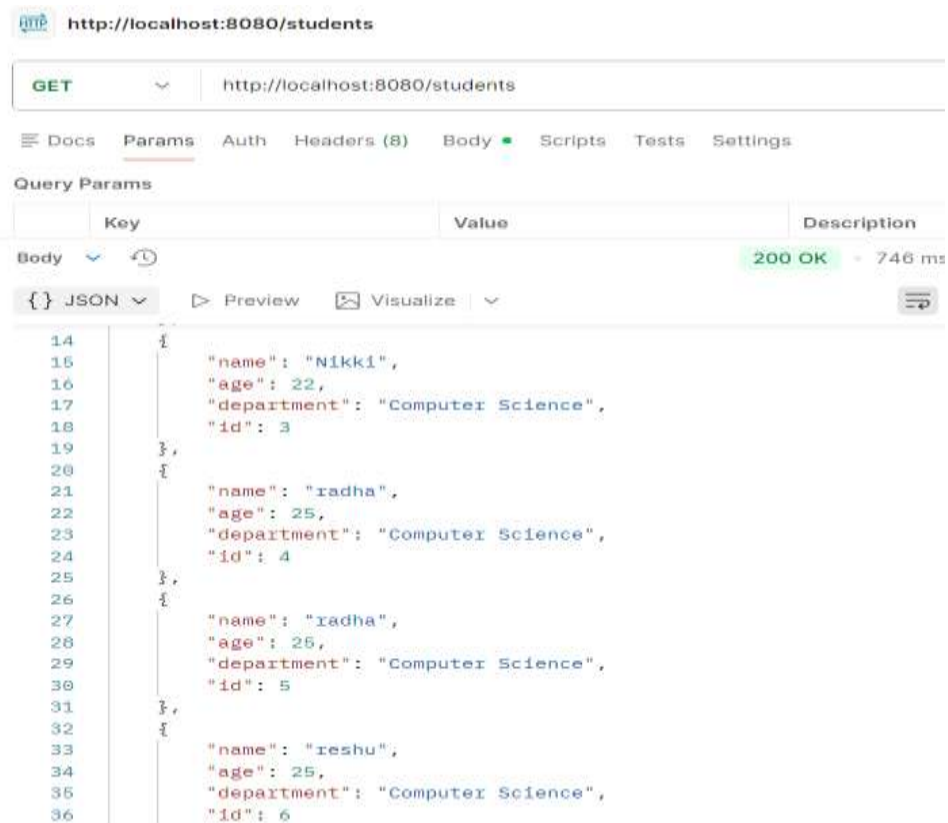
```

```

public student createStudent(@RequestBody student student) {
    return service.saveStudent(student);
} @GetMapping
public List<student> getAllStudents() {
    return service.getAllStudents();
} @GetMapping("/{id}")
public student getStudentById(@PathVariable Long id) {
    return service.getStudentById(id);
} @PutMapping("/{id}")
public student updateStudent(@PathVariable Long id,
                             @RequestBody student student) {
    return service.updateStudent(id, student);
} @DeleteMapping("/{id}")
public void deleteStudent(@PathVariable Long id) {
    service.deleteStudent(id);
} @GetMapping("/department/{department}")
public List<student> getByDepartment(@PathVariable String department) {
    return service.getStudentsByDepartment(department);
} @GetMapping("/age/{age}")
public List<student> getByAge(@PathVariable int age) {
    return service.getStudentsByAgeGreaterThan(age);
} @GetMapping("/sort")
public List<student> sortStudents() {
    return service.getStudentsSortedByName();
} @GetMapping("/page")
public Object paginateStudents(@RequestParam int page,
                               @RequestParam int size) {
    return service.getStudentsWithPagination(page, size);
}

```

Output :



The screenshot displays a REST client interface for a GET request to `http://localhost:8080/students`. The response is a `200 OK` status with a response time of `746 ms`. The response body is shown in JSON format, containing an array of four student records.

```
{
  "name": "Nikki",
  "age": 22,
  "department": "Computer Science",
  "id": 3
},
{
  "name": "radha",
  "age": 25,
  "department": "Computer Science",
  "id": 4
},
{
  "name": "radha",
  "age": 25,
  "department": "Computer Science",
  "id": 5
},
{
  "name": "reshu",
  "age": 25,
  "department": "Computer Science",
  "id": 6
}
```

Result

The Data Access Layer was successfully implemented using Spring Data JPA, enabling efficient retrieval of student records using custom query methods, along with sorting and pagination for optimized data handling.